

EX-1

```

# Step 1: Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.datasets import load_iris # Import load_iris

# Step 2: Load Dataset
# Load the Iris dataset from sklearn
iris = load_iris()
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df['species'] = iris.target_names[iris.target]

# Rename columns to match the original code
df = df.rename(columns={'sepal length (cm)': 'sepal.length', 'sepal width (cm)': 'sepal.width', 'petal length (cm)': 'petal.length'})

# -----
# ♦ UNIVARIATE REGRESSION
# X = sepal.length, y = petal.length
# -----
X_uni = df[['sepal.length']]
y_uni = df['petal.length']

model_uni = LinearRegression()
model_uni.fit(X_uni, y_uni)

y_pred_uni = model_uni.predict(X_uni)

# Plot
plt.scatter(X_uni, y_uni)
plt.plot(X_uni, y_pred_uni)
plt.title("Univariate Regression")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")
plt.show()

print("Univariate R^2:", model_uni.score(X_uni, y_uni))

# -----
# ♦ BIVARIATE REGRESSION
# X = sepal.length, sepal.width
# -----
X_bi = df[['sepal.length', 'sepal.width']]
y_bi = df['petal.length']

model_bi = LinearRegression()
model_bi.fit(X_bi, y_bi)

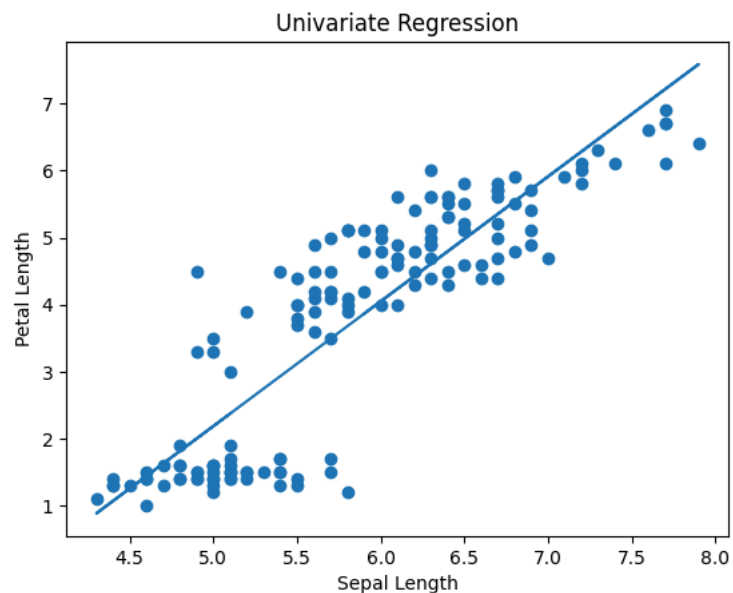
print("Bivariate R^2:", model_bi.score(X_bi, y_bi))

# -----
# ♦ MULTIVARIATE REGRESSION
# X = all features
# -----
X_multi = df[['sepal.length', 'sepal.width', 'petal.width']]
y_multi = df['petal.length']

model_multi = LinearRegression()
model_multi.fit(X_multi, y_multi)

print("Multivariate R^2:", model_multi.score(X_multi, y_multi))

```



Univariate R^2 : 0.759954645772515
 Bivariate R^2 : 0.8676860089345193
 Multivariate R^2 : 0.9680117693912218

EX-2

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load dataset (using sample data as requested, without read_csv)
x = np.array([2500, 2600, 2700, 2800, 2900, 3000, 3100, 3200, 3300, 3400, 3500, 3600]) # Sample Head Size(cm^3)
y = np.array([1300, 1350, 1400, 1450, 1500, 1550, 1600, 1650, 1700, 1750, 1800, 1850]) # Sample Brain Weight(grams)

# ---- Least Squares Formula ----
x_mean = np.mean(x)
y_mean = np.mean(y)

m = np.sum((x - x_mean)*(y - y_mean)) / np.sum((x - x_mean)**2)
c = y_mean - m*x_mean

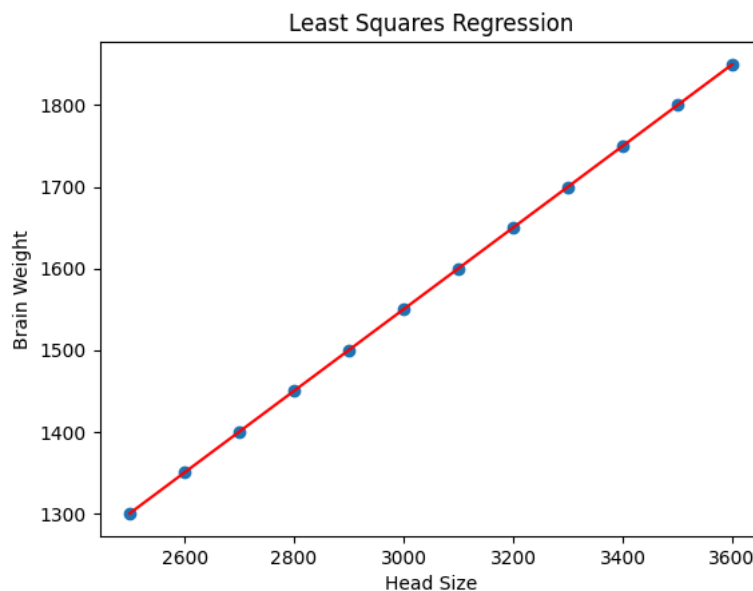
print("Slope (m):", m)
print("Intercept (c):", c)

# Prediction line
y_pred = m*x + c

# Plot
plt.scatter(x, y)
plt.plot(x, y_pred, color='red') # Added color for clarity
plt.xlabel("Head Size")
plt.ylabel("Brain Weight")
plt.title("Least Squares Regression")
plt.show()

# R^2 Score
r2 = 1 - (np.sum((y - y_pred)**2) / np.sum((y - y_mean)**2))
print("R^2 Score:", r2)
```

Slope (m): 0.5
Intercept (c): 50.0



EX-3

```
# Step 1: Import Libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
import numpy as np # Import numpy for sample data generation

# Step 2: Load Dataset (using sample data)
# Creating a synthetic dataset as 'suv_data.csv' was not found
np.random.seed(42) # for reproducibility
sample_data = {
    'Age': np.random.randint(18, 65, 400),
    'EstimatedSalary': np.random.randint(15000, 150000, 400),
    'Purchased': np.random.randint(0, 2, 400) # 0 for No, 1 for Yes
}
data = pd.DataFrame(sample_data)

# Step 3: Features & Target
X = data[['Age', 'EstimatedSalary']].values
y = data['Purchased'].values

# Step 4: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)

# Step 5: Feature Scaling
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# Step 6: Train Model
model = LogisticRegression()
model.fit(X_train, y_train)

# Step 7: Prediction
y_pred = model.predict(X_test)

# Step 8: Evaluation
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
Confusion Matrix:
[[28 16]
 [20 16]]
Accuracy: 0.55
Classification Report:
              precision    recall  f1-score   support

0               0.58         0.64         0.61         44
```

1	0.50	0.44	0.47	36
accuracy			0.55	80
macro avg	0.54	0.54	0.54	80
weighted avg	0.55	0.55	0.55	80

EX-4

```
import numpy as np

# Step 1: Input (AND gate example)
X = np.array([[0,0],
              [0,1],
              [1,0],
              [1,1]])

y = np.array([[0],[0],[0],[1]])

# Step 2: Initialize weights & bias
weights = np.zeros((2,1))
bias = 0
lr = 0.1

# Step 3: Activation function
def sigmoid(x):
    return 1/(1 + np.exp(-x))

# Step 4: Training
for epoch in range(1000):
    z = np.dot(X, weights) + bias
    y_pred = sigmoid(z)

    error = y - y_pred

    # update weights
    weights += lr * np.dot(X.T, error)
    bias += lr * np.sum(error)

# Step 5: Output
print("Weights:\n", weights)
print("Bias:\n", bias)

# Step 6: Prediction
pred = sigmoid(np.dot(X, weights) + bias)
print("Predictions:\n", pred.round())
```

```
Weights:
[[5.5699554]
 [5.5699554]]
Bias:
-8.534581991053331
Predictions:
[[0.]
 [0.]
 [0.]
 [1.]]
```

EX-5

```
# Step 1: Import Libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np # Import numpy for sample data generation

# Step 2: Load Dataset (using sample data)
# Creating a synthetic dataset as 'banknotes.csv' was not found
np.random.seed(42) # for reproducibility
sample_data = {
    'Variance': np.random.rand(1000) * 5 - 2.5, # Example range for variance
    'Skewness': np.random.rand(1000) * 10 - 5, # Example range for skewness
    'Curtosis': np.random.rand(1000) * 10 - 5, # Example range for curtosis
    'Entropy': np.random.rand(1000) * 5 - 2.5, # Example range for entropy
    'Class': np.random.randint(0, 2, 1000) # 0 for authentic, 1 for forged
}
data = pd.DataFrame(sample_data)

# Step 3: Features & Target
X = data.drop("Class", axis=1)
```

```

y = data["Class"]

# Step 4: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)

# Step 5: Create Model
model = MLPClassifier(hidden_layer_sizes=(5,), max_iter=500)

# Step 6: Train
model.fit(X_train, y_train)

# Step 7: Predict
y_pred = model.predict(X_test)

# Step 8: Evaluation
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))

```

```

Confusion Matrix:
[[53 57]
 [36 54]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.60	0.48	0.53	110
1	0.49	0.60	0.54	90
accuracy			0.54	200
macro avg	0.54	0.54	0.53	200
weighted avg	0.55	0.54	0.53	200

EX-6

```

# Step 1: Import Libraries
import pandas as pd
from sklearn import svm
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np # Import numpy for sample data generation

# Step 2: Load Dataset (using sample data)
# Creating a synthetic dataset as 'muffins.csv' was not found
np.random.seed(42) # for reproducibility
sample_size = 100
sample_data = {
    'Sugar': np.random.uniform(0, 100, sample_size), # Example range for sugar
    'Flour': np.random.uniform(0, 100, sample_size), # Example range for flour
    'Type': np.random.choice(['Muffin', 'Cupcake'], sample_size) # Randomly assign Muffin or Cupcake
}
df = pd.DataFrame(sample_data)

# Step 3: Features & Labels
X = df[['Sugar', 'Flour']].values
y = df['Type'].apply(lambda x: 0 if x == 'Muffin' else 1).values

# Step 4: Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)

# Step 5: Train SVM
model = svm.SVC(kernel='linear')
model.fit(X_train, y_train)

# Step 6: Predict
y_pred = model.predict(X_test)

# Step 7: Evaluate
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))

```

```

Confusion Matrix:
[[ 0 11]
 [ 0  9]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.00	0.00	0.00	11
1	0.45	1.00	0.62	9
accuracy			0.45	20
macro avg	0.23	0.50	0.31	20

weighted avg 0.20 0.45 0.28 20

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

EX-7

```
# Step 1: Import Libraries
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt

# Step 2: Load Dataset
iris = load_iris()
X = iris.data
y = iris.target

# Step 3: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)

# Step 4: Train Model
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# Step 5: Predict
y_pred = model.predict(X_test)

# Step 6: Evaluation
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))

# Step 7: Plot Tree
plt.figure(figsize=(8,6))
plot_tree(model, filled=True)
plt.title("Decision Tree")
plt.show()
```

Confusion Matrix:

```
[[11  0  0]
 [ 0 13  0]
 [ 0  0  6]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	6
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Decision Tree



EX-8

```
# Step 1: Import Libraries
from sklearn.datasets import load_iris
from sklearn.ensemble import AdaBoostClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 2: Load Dataset
iris = load_iris()
X = iris.data
y = iris.target

# Step 3: Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)

# Step 4: Train AdaBoost Model
model = AdaBoostClassifier(n_estimators=50, random_state=0)
model.fit(X_train, y_train)

# Step 5: Predict
y_pred = model.predict(X_test)

# Step 6: Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

Accuracy: 0.9666666666666667

Confusion Matrix:

```
[[11  0  0]
 [ 0 13  0]
 [ 0  1  5]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11
1	0.93	1.00	0.96	13
2	1.00	0.83	0.91	6

accuracy			0.97	30
macro avg	0.98	0.94	0.96	30
weighted avg	0.97	0.97	0.97	30

EX-9

```
# Step 1: Import Libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Step 2: Load Data
iris = load_iris()
X = iris.data
y = iris.target

# Step 3: Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)

# Step 4: Train Model
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

# Step 5: Predict
y_pred = model.predict(X_test)

# Step 6: Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Accuracy: 0.9666666666666667
Confusion Matrix:
[[11 0 0]
[0 12 1]
[0 0 6]]

EX-10

```
# Step 1: Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
```